1、题目中所有参数初始化为1，假设W_q, W_k, W_v为3x3矩阵，全为1：

$$W_q = W_k = W_v = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$q^1 = W_q a^1 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix}$$

类似地，$q^i = k^i = v^i = \begin{bmatrix} 3i \\ 3i \\ 3i \end{bmatrix}$

∴ $q^1 = k^1 = v^1 = [3, 3, 3]^T$ $q^2 = k^2 = v^2 = [6, 6, 6]^T$

$q^3 = k^3 = v^3 = [9, 9, 9]^T$ $q^4 = k^4 = v^4 = [12, 12, 12]^T$

$S_{ij} = (q^i)^T k^j$ $a_{ij} = \frac{\exp(S_{ij})}{\sum_{k=1}^{n} \exp(S_{ik})}$ $b^i = \sum_{j=1}^{n} a_{ij} v^j$

$S_{1j} = (q^1)^T k^j$

$S_{11} = (q^1)^T k^1 = [3, 3, 3] \cdot \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix} = 27$

$S_{12} = (q^1)^T k^2 = [3, 3, 3] \cdot \begin{bmatrix} 6 \\ 6 \\ 6 \end{bmatrix} = 54$

$S_{13} = (q^1)^T k^3 = [3, 3, 3] \cdot \begin{bmatrix} 9 \\ 9 \\ 9 \end{bmatrix} = 81$

$S_{14} = (q^1)^T k^4 = [3, 3, 3] \cdot \begin{bmatrix} 12 \\ 12 \\ 12 \end{bmatrix} = 108$

$a_{11}=\frac{e^{27}}{e^{27}+e^{54}+e^{81}+e^{108}}$ $a_{12}=\frac{e^{54}}{e^{27}+e^{54}+e^{81}+e^{108}}$ $a_{13}=\frac{e^{81}}{e^{27}+e^{54}+e^{81}+e^{108}}$

$a_{14}=\frac{e^{108}}{e^{27}+e^{54}+e^{81}+e^{108}}$ a_{14}远大于其他几个

$a_{11}\approx 0$ $a_{12}\approx 0$ $a_{13}\approx 0$ $a_{14}\approx 1$

$b^1=\sum_{j=1}^{4} a_{1j}V^j = V^4 = \begin{bmatrix}12\\12\\12\end{bmatrix}$

则b为$\begin{bmatrix}12\\12\\12\end{bmatrix}$

2. a) 每层后的图像/特征图大小是多少？

① $W_{out}=\left\lfloor\frac{W_{in}-F+2P}{S}\right\rfloor+1$

$W_{in}=13$ $F=4$ $P=0$ $S=1$

$\therefore W_{out}=\frac{13-4+0}{1}+1=10$

通道数 3

则卷积层输出 $10\times10\times3$

② $W_{out}=\left\lfloor\frac{W_{in}}{S}\right\rfloor$

$W_{in}=10$ $S=2$

$\therefore W_{out}=5$

通道数 3

则最大池化层输出 $5\times5\times3$

③ ReLU不改变维度. 仅对每个元素用 $ReLU(x) = \max(0, x)$

ReLU层输出 $5 \times 5 \times 3$

④ 全连接层输出： 4×1.

(a) 卷积层需要学习多少权重

每个滤波器：

$$F_{height} \times F_{width} \times 输入通道数 = 4 \times 4 \times 1 = 16$$

总：

$$3 \times 16 = 48$$

(c) 前向传播中执行了多少次ReLU

ReLU次数等于ReLU层输入的元素数

$$5 \times 5 \times 3 = 75$$

(d) 整个网络要学习多少权重

由(b)，卷积层 48个

全连接层输入：$5 \times 5 \times 3 = 75$

输出：4

输入 × 输出 $= 75 \times 4 = 300$

总：$300 + 48 = 348$

3、如图所示：

```python
import numpy as np

logits = {
    'dog': 2.1,
    'cat': 1.9,
    'you': 3.3,
    'the': -1.2,
    'a': 0.0,
    'hello': -2.9,
    'fish': 1.9,
    'apple': -0.2,
    'banana': 0.8,
    'egg': 1.1
}

tokens = list(logits.keys())
logit_values = np.array([logits[t] for t in tokens])

def softmax(x):
    exp_x = np.exp(x - np.max(x))  # 防止溢出
    return exp_x / np.sum(exp_x)

probs = softmax(logit_values)
prob_dict = {t: p for t, p in zip(tokens, probs)}
sorted_probs = sorted(prob_dict.items(), key=lambda x: x[1], reverse=True)

print("a) Softmax 概率（降序）: ")
for token, prob in sorted_probs:
    print(f"{token}: {prob:.6f}")
```

```
a) Softmax 概率（降序）:
you: 0.483677
dog: 0.145681
cat: 0.119273
fish: 0.119273
egg: 0.053593
banana: 0.039703
a: 0.017840
apple: 0.014606
the: 0.005373
hello: 0.000982
```

```python
T = 2.0
probs_temp = softmax(logit_values / T)
prob_dict_temp = {t: p for t, p in zip(tokens, probs_temp)}
sorted_probs_temp = sorted(prob_dict_temp.items(), key=lambda x: x[1], reverse=True)

print("\nb) 温度 T=2.0 概率（降序）：")
for token, prob in sorted_probs_temp:
    print(f"{token}: {prob:.6f}")
```

```
b) 温度 T=2.0 概率（降序）：
you: 0.271914
dog: 0.149230
cat: 0.135029
fish: 0.135029
egg: 0.090512
banana: 0.077905
a: 0.052221
apple: 0.047252
the: 0.028660
hello: 0.012250
```

```python
top_k = 5
top_k_items = sorted_probs_temp[:top_k]
top_k_tokens = [item[0] for item in top_k_items]
top_k_probs = [item[1] for item in top_k_items]
top_k_probs_normalized = np.array(top_k_probs) / np.sum(top_k_probs)
top_k_dict = {t: p for t, p in zip(top_k_tokens, top_k_probs_normalized)}
sorted_top_k = sorted(top_k_dict.items(), key=lambda x: x[1], reverse=True)

print("\nc) Top_k=5 概率（降序）：")
for token, prob in sorted_top_k:
    print(f"{token}: {prob:.6f}")
```

```
c) Top_k=5 概率（降序）：
you: 0.347844
dog: 0.190901
cat: 0.172734
fish: 0.172734
egg: 0.115787
```

```python
top_p = 0.5
cumsum = np.cumsum([p for _, p in sorted_top_k])
selected_idx = np.where(cumsum >= top_p)[0][0] + 1
top_p_items = sorted_top_k[:selected_idx]
top_p_tokens = [item[0] for item in top_p_items]
top_p_probs = [item[1] for item in top_p_items]
top_p_probs_normalized = np.array(top_p_probs) / np.sum(top_p_probs)
top_p_dict = {t: p for t, p in zip(top_p_tokens, top_p_probs_normalized)}
sorted_top_p = sorted(top_p_dict.items(), key=lambda x: x[1], reverse=True)

print("\nd) Top_p=0.5 概率（降序）：")
for token, prob in sorted_top_p:
    print(f"{token}: {prob:.6f}")
```

```
d) Top_p=0.5 概率（降序）：
you: 0.645656
dog: 0.354344
```